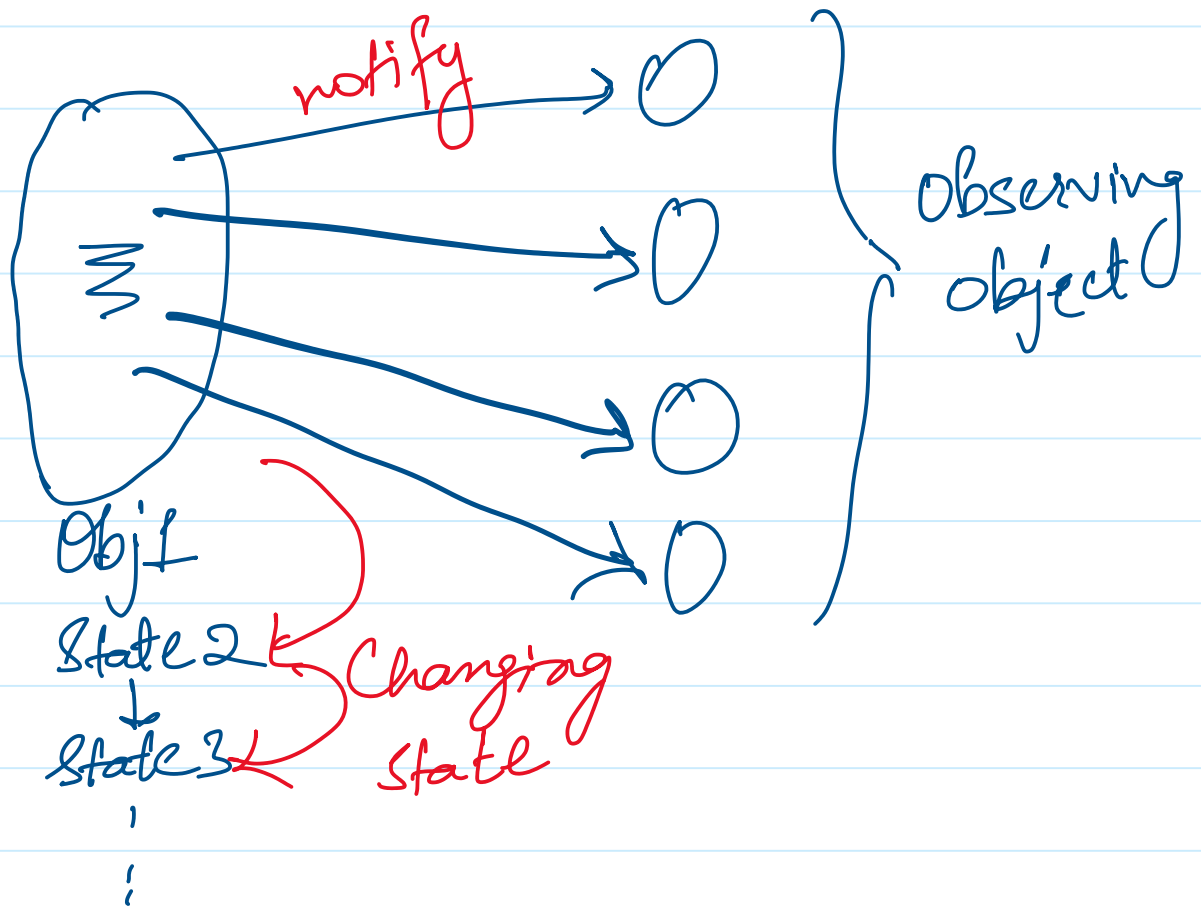


When you subscribe to one channel then how you get notification when they upload any video or content.

How your account and that channel account interact - This is Observer Design Pattern, how both communicate.

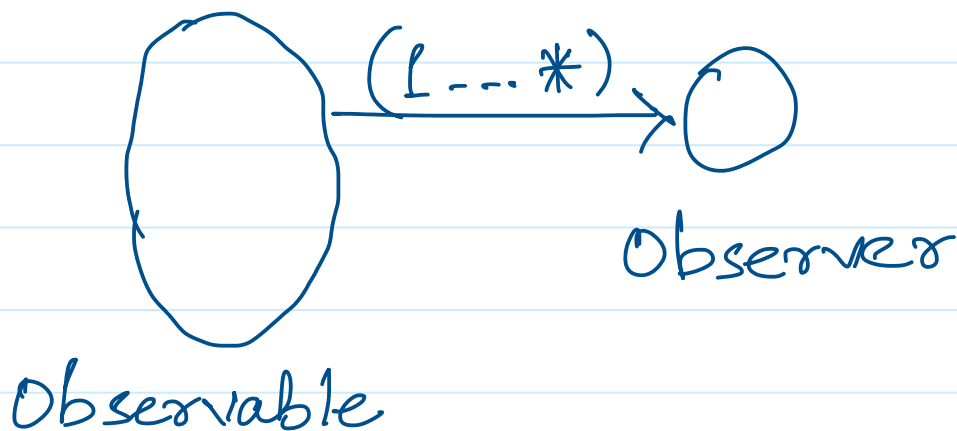
This Design Pattern solves very specific problem i.e - Suppose 2 objects are there and whenever any one object change then other objects wants to know when it changes and what was its value when it changes, and changing object wants to communicate other object when it changes.

Observer Design Pattern.



when Obj1 changes state than it will notify other objects

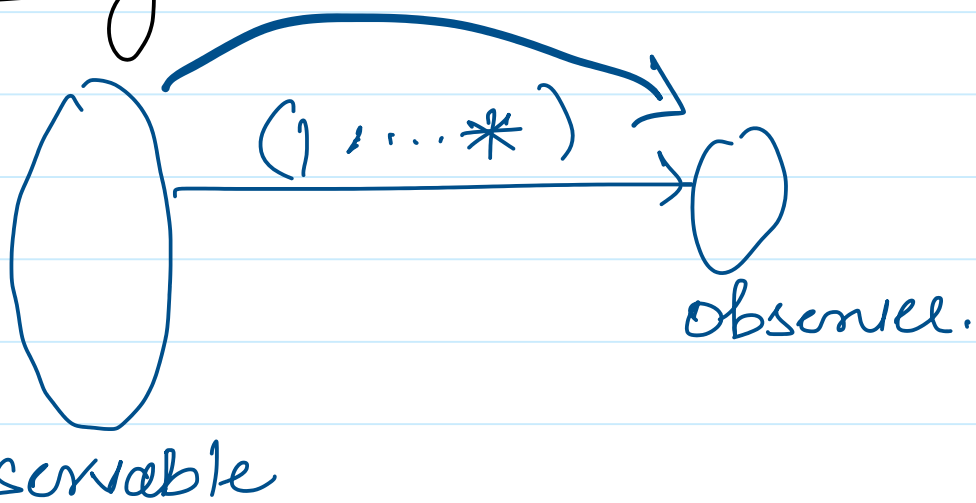
Polling



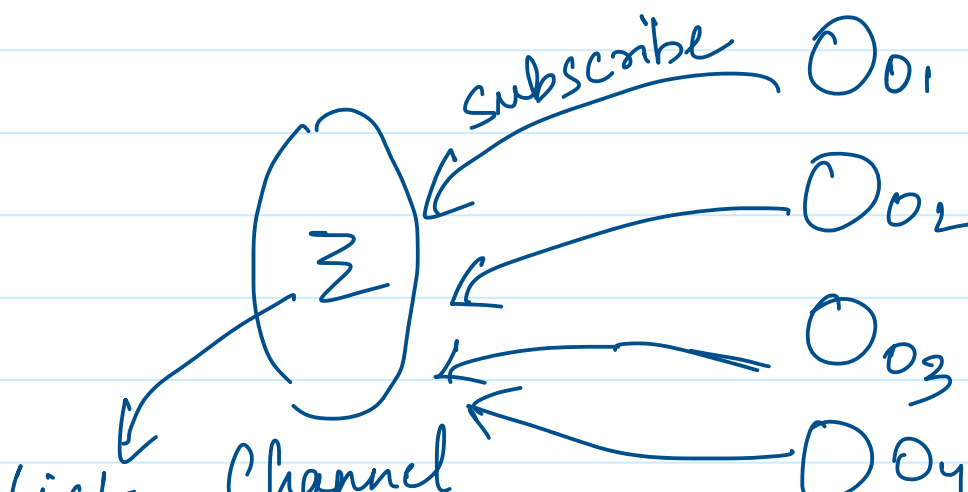
Observer keeps on asking observable
"is my observable changed?"

Observer keeps on asking observable "is your value changed?" at some frequency. At some time observer will get to know that observable value changed.
→ very Bad way

Pushing



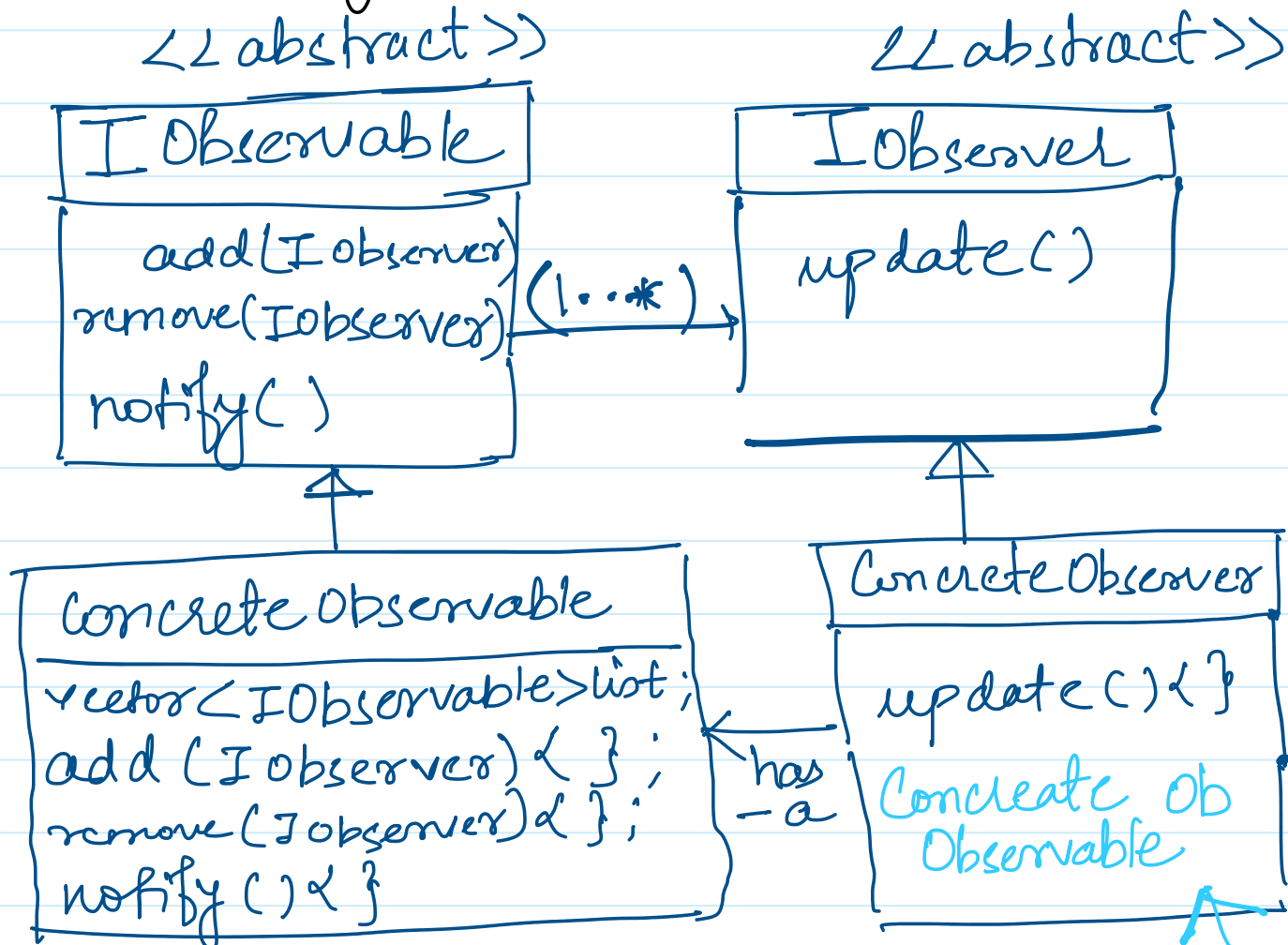
In this it is responsibility of observable to tell his observers whenever its values/state changes.
Now,



list ↙ Channel ← Observable

Observable will have a list of object who have subscribed, so whenever it changes state it will notify all his subscribers.

UML Diagram.



`add` → To add `IObservable` in the list
`remove` → To remove `IObservable` from the list

`notify` → To notify the Observer that its state has changed.

`update()` → it is called when the state of observable changes

notify will call update method for all Observer in the list

add is like subscribe & remove is like un-subscribe

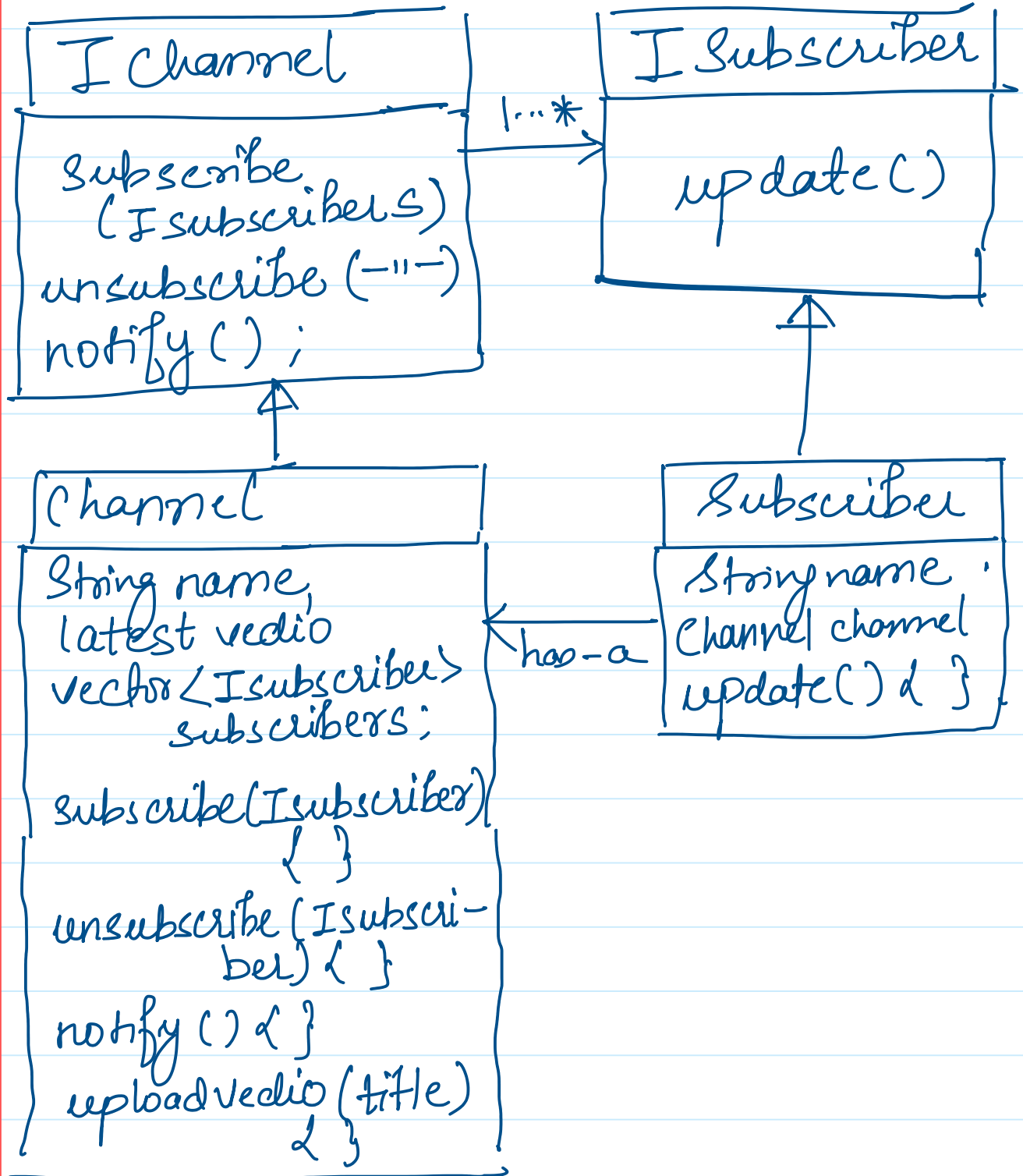
In Notify(), it will loop through each Observer in list and call its update method.

ConcreteObserver will also have a object of ConcreteObservable b/c Observer should also know which observable's state/value has changed. By doing this observer know whose value/state it has to check.

Defination :-

Defines a one-to-many relationship b/w objects so that when one object changes state, all of its dependents are notified and update automatically.

Youtube Example



When new video comes uploadVideo() call → internally it calls notify()

call → internally it calls notify()

In notify it loop through the list of subscribers and calls their update() method.

Now subscriber has the instance of "channel" to which it has subscribed. By using this "channel" instance subscriber can get all the info about latest video uploaded, description.

Code → see Notes...

Our Observer Design Pattern is
breaking SRP

↓
Single-Responsibility Principle. ~~X~~

How?

A Channel Class is doing - notify(),
uploadVideo(), getVideo(), subscribe()
unsubscribe().

So, there is trade-off

Use - Cases.

1. Notification Service.
2. News-feed Service
3. Event - handling in Programming
↓
like in javascript - onclick, onhover
etc.